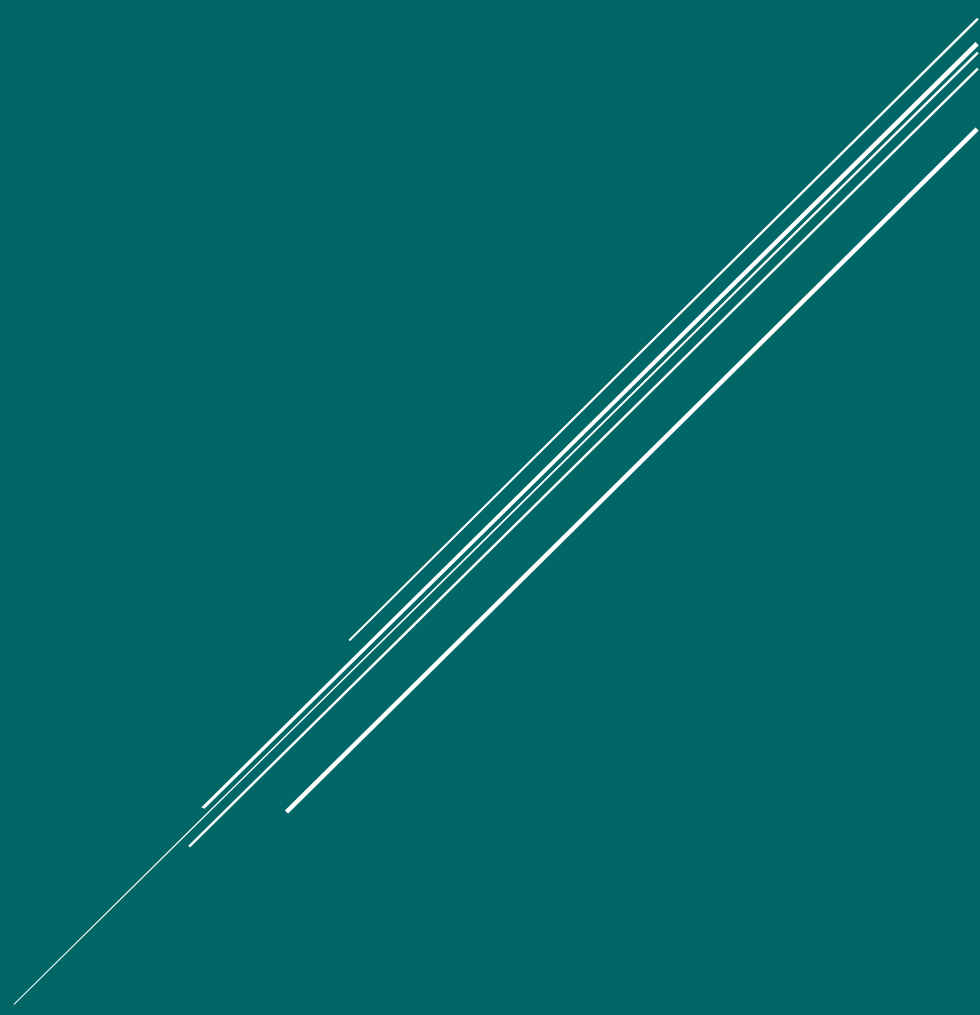




VDA

Librería para Visualización de Datos

García Aguilar Luis Alberto

Contenido

Introducción	2
Introducción y objetivos.....	3
Configuración inicial	4
Configuración de la biblioteca.....	5
Métodos	6
View Port	7
X Axis	9
Y Axis	10
X Axis from array	11
Y Axis from array	
DotPlot	13
LinePlot.....	14
BarPlot.....	15
bubblePlot	16
MapDotPlot	17
HeatMap.....	19
Casos de uso	21
View Port	22
Eje X.....	26
Eje Y	29
Eje X a partir de un arreglo de datos.....	32
Eje Y a partir de un arreglo de datos.....	36
Gráfica de puntos	40
Gráfica de líneas	43
Gráfica de barras	46
Gráfica de burbujas	49
Mapa de puntos	53
Mapa de burbujas	56
Mapa de color (República Mexicana).....	59
Mapa de color (USA)	62
Referencias.....	65



Introducción

Introducción y objetivos

Este documento pretende ser una guía para el uso de nuestra biblioteca de visualización de datos en JavaScript, Vda. Esta guía está diseñada para proporcionar una referencia completa a los métodos disponibles en la biblioteca, facilitando a los desarrolladores la creación de gráficos y visualizaciones interactivas en un entorno web utilizando elementos de canvas.

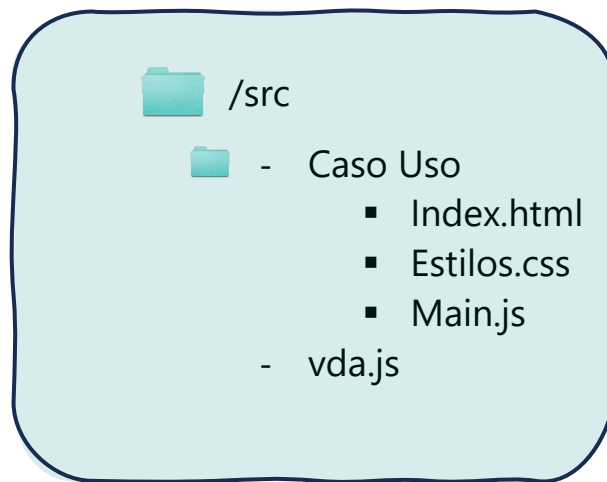
Se pretende ofrecer una descripción de cada método proporcionado en la biblioteca, incluyendo su propósito, entradas, salidas y casos de uso. La biblioteca tiene como objetivo facilitar al desarrollador el uso de canvas para poder crear una variedad de gráficos y visualizaciones, tales como gráficos de líneas, gráficos de barras, gráficos de burbujas, entre otros.



Configuración inicial

Configuración de la biblioteca

Esta guía considera la siguiente estructura para cada caso de uso:



Para hacer uso de la biblioteca y sus métodos es necesario importarla dentro del proyecto:

Por ejemplo, en el archivo `main.js` se deberá agregar la siguiente línea de código al inicio del archivo:

```
import {initViewport, drawXAxis} from './lib/vda.js';
```

Entre corchetes se indican los métodos a utilizar y entre comillas se indica la ruta del archivo `vda.js`

También es posible importar la librería completa de la siguiente manera:

```
import * as vda from './vda.js';
```



Métodos

View Port

Uso



El método `initViewport` se encarga de inicializar un espacio de trabajo en un elemento canvas. Este método ajusta el tamaño del canvas y configura las coordenadas para trabajar con una matriz inversa, facilitando la representación gráfica de datos en el canvas.

Entradas

- **canvasId** (string): El ID del canvas en el documento HTML.
- **width** (number): Ancho para el canvas.
- **height** (number): Altura para el canvas.

Salidas

context (`CanvasRenderingContext2D`): Contexto del canvas, configurado para trabajar con el viewport ajustado.

Método { }

```
initViewport (canvasId, width,  
height) {  
  
    ...  
  
    ...  
  
    return context;  
}
```


Texto

Uso



El método `drawText` permite insertar un texto en el canvas en base a una posición y ángulo indicados.

Entradas



- **ctx** (CanvasRenderingContext2D): Contexto del canvas donde se dibujará.
- **text** (String): Texto a mostrar.
- **x** (number): Posición X en el canvas donde se insertará el texto.
- **y** (number): Posición Y en el canvas donde se insertará el texto.
- **size** (number): Tamaño del texto
- **color** (string): Color.
- **angle** (number): Ángulo del texto.

Salidas



No retorna valor. Dibujará directamente el texto.

Método { }

```
drawText(ctx, text, x, y, size,  
color, angle)  
{  
  
    ...  
}
```

Eje X

Uso



El método `drawXAxis` dibuja una línea de eje con marcas (ticks) distribuidas uniformemente a lo largo del eje. Las marcas se generan para un rango de valores especificado por el usuario (`start`, `end`) y se espacian según un paso dado (`step`).

Entradas



- **Context** (CanvasRenderingContext2D): Contexto del canvas donde se dibujará.
- **startX** (number): Valor inicial del rango del eje X.
- **endX** (number): Valor de fin del rango del eje X.
- **xPos** (number): Posición X en el canvas donde se comenzará a dibujar el eje X.
- **yPos** (number): Posición Y en el canvas donde se comenzará a dibujar el eje X.
- **step**: Paso entre cada marca (tick) en el eje X.
- **color** (string): Color.
- **labelSpace** (number): Espacio entre etiquetas y el eje X.

Salidas



No retorna valor. Dibujará directamente el eje X.

Método { }

```
drawXAxis (context, startX, endX,  
xPos, yPos, step, color, labelSpace)  
{  
  
    ...  
  
}
```

Eje Y

Uso



El método `drawYAxis` dibuja una línea de eje Y con marcas (ticks) distribuidas uniformemente a lo largo del eje. Las marcas se generan para un rango de valores especificado por el usuario (`start`, `end`) y se espacian según un paso dado (`step`).

Entradas



- **Context** (CanvasRenderingContext2D): Contexto del canvas donde se dibujará.
- **startX** (number): Valor inicial del rango del eje Y.
- **endX** (number): Valor de fin del rango del eje Y.
- **xPos** (number): Posición X en el canvas donde se comenzará a dibujar el eje Y.
- **yPos** (number): Posición Y en el canvas donde se comenzará a dibujar el eje Y.
- **step**: Paso entre cada marca (tick) en el eje Y.
- **color** (string): Color.
- **labelSpace** (number): Espacio entre etiquetas y el eje Y.

Salidas



No retorna valor. Dibujará directamente el eje Y.

Método { }

```
DrawYAxis (context, startY, endY,  
xPos, yPos, step, color, labelSpace)  
{  
  
    ...  
  
}
```

Eje X a partir de un arreglo de datos

Uso



El método `drawXAxisWithIntervals` dibuja una línea de eje X en un canvas a partir de valores definidos en un arreglo proporcionado.

Entradas



- **Context** (`CanvasRenderingContext2D`): Contexto del canvas.
- **xValues** (`Array[Object]`): Array de datos 1xn que contiene los valores para las marcas del eje.
- **xPos** (`number`): Posición X inicial para el eje.
- **yPos** (`number`): Posición Y inicial para el eje.
- **xLabelTextAngle** (`number`): Ángulo de las etiquetas en el eje.
- **color** (`string`): Color.
- **labelSpace** (`number`): Espacio entre etiquetas y el propio eje.
- **canvasPadding** (`number`): Padding del canvas.
- **Interval** (`number`): -1: Intervalo calculado en base al rango; >0: intervalo usando el valor indicado; 0: se ocupan todos los valores del arreglo.

Salidas



No retorna valor. Dibujará directamente el eje X.

Método { }

```
drawXAxisWithIntervals(context,  
xValues , xPos, yPos, , color,  
labelSpace, canvasPadding, 0)
```

```
{      ...      }
```

Eje Y a partir de un arreglo de datos

Uso



El método `drawYAxisWithIntervals` dibuja una línea de eje Y en un canvas a partir de valores definidos en un arreglo proporcionado.

Entradas



- **Context** (CanvasRenderingContext2D): Contexto del canvas.
- **yValues** (Array[Object]): Array de datos 1xn que contiene los valores para las marcas del eje.
- **xPos** (number): Posición X inicial para el eje.
- **yPos** (number): Posición Y inicial para el eje.
- **color** (string): Color.
- **labelSpace** (number): Espacio entre etiquetas y el propio eje.
- **canvasPadding** (number): Padding del canvas.
- **Interval** (number): -1: Intervalo calculado en base al rango; >0: intervalo usando el valor indicado; 0: se ocupan todos los valores del arreglo.

Salidas



No retorna valor. Dibujará directamente el eje Y.

Método { }

```
drawYAxisWithIntervals(context,  
yValues , xPos, yPos, color,  
labelSpace, canvasPadding, interval)  
{  
  
    ...  
}
```

Gráfica de puntos

Uso



El método `drawDotPlot` dibuja una gráfica de puntos basada en un conjunto de datos proporcionados mediante un archivo csv. Cada punto de datos se representa como un círculo en el gráfico.

Entradas



- **canvas** (Canvas): Canvas en uso.
- **context** (CanvasRenderingContext2D): Contexto.
- **csvFilePath** (String): Ruta al archivo de datos.
- **xColumnName** (String): Nombre para la columna X.
- **yColumnName** (String): Nombre para la columna Y.
- **infoColumnNames** (Array[String]): Arreglo con el nombre de las columnas que serán presentadas como información al pasar el cursor sobre el gráfico.
- **color** (String): Color de los puntos.
- **filledCircles** (Boolean): True para círculos rellenos.
- **pointRadius** (number): Radio de los puntos.
- **canvasPadding** (number): Padding del canvas.
- **axesProperties** (Object, opt): Objeto que contiene las propiedades definidas para los ejes (Ver caso práctico).

Salidas



No retorna valor.

Método { }

```
drawDotPlot (canvas, context,  
csvFilePath, xColumnName,  
yColumnName, infoColumnNames, color,  
filledCircles, pointRadius,  
canvasPadding, axesProperties)  
  
{  
  
    ...  
  
}
```

Gráfica de líneas

Uso



El método `drawLinePlot` dibuja una gráfica de línea basada en un conjunto de datos proporcionado mediante un archivo csv. Cada punto de datos se conecta mediante líneas rectas, creando una representación gráfica de la serie de datos.

Entradas



- **canvas** (Canvas): Canvas.
- **context** (CanvasRenderingContext2D): Contexto del canvas.
- **filePath** (String): Ruta al archivo de datos.
- **xColumnName** (String): Nombre de la columna de datos a usar para el eje X.
- **yColumnName** (String): Nombre de la columna de datos a usar para el eje Y.
- **infoColumnNames** (Array[String]): Arreglo con el nombre de las columnas que serán presentadas como información al pasar el cursor sobre el gráfico.
- **color** (String): Color de los puntos.
- **filledCircles** (Boolean): True para círculos rellenos.
- **pointRadius** (number): Radio de los puntos.
- **lineWidth** (number): Grosor de la línea.
- **canvasPadding** (number): Padding del canvas.
- **axesProperties** (Object, opt): Objeto que contiene las propiedades definidas para los ejes (Ver caso práctico).

Salidas



No retorna valor.

Método { }

```
drawLinePlot(canvas, context, filePath,
xColumnName, yColumnName, infoColumnNames,
color, filledCircles, pointRadius,
lineWidth, canvasPadding, axesProperties)
```

```
{ ... }
```

Gráfica de barras

Uso



El método `drawBarPlot` dibuja una gráfica de barras basada en un conjunto de datos proporcionado en un archivo csv. Cada barra representa una categoría y se dibuja en el canvas con una altura proporcional al conteo de ocurrencias en cada categoría.

Entradas



- **canvas** (Canvas): Canvas.
- **context**: Contexto del canvas.
- **filePath** (String): Ruta al archivo de datos.
- **xColumnName** (String): Nombre de la columna de categorías.
- **canvasPadding** (number): Padding del canvas.

Salidas



No retorna valor.

Método { }

```
drawBarPlot (canvas, context,  
filePath, xColumnName,  
canvasPadding)  
{  
    ...  
    ...  
}
```


Gráfica de burbujas

Uso



El método `drawBubblePlot` dibuja una gráfica de burbujas basada en un conjunto de datos en un archivo csv. Cada burbuja se representa como un círculo en el gráfico, con un radio proporcional a un valor específico indicado.

Entradas



- **canvas** (Canvas): Canvas.
- **context** (CanvasRenderingContext2D): contexto.
- **filePath** (String): Ruta al archivo de datos.
- **xColumnName** (String): Nombre de la columna de datos a usar para el eje X.
- **yColumnName** (String): Nombre de la columna de datos a usar para el eje Y.
- **sizeColumnName** (String): Nombre de la columna que indicará el tamaño de las burbujas.
- **minSize** (number): Tamaño mínimo de burbuja.
- **maxSize** (number): Tamaño máximo de burbuja.
- **infoColumnNames** (Array[String]): Arreglo con el nombre de las columnas que se desplegarán como información al pasar el cursor sobre el gráfico.
- **color** (String): Color de los puntos.
- **canvasPadding** (number): Padding del canvas.
- **axesProperties** (Object, opt): Objeto que contiene las propiedades definidas para los ejes (Ver caso práctico).

Salidas



No retorna valor.

Método { }

```
drawBubblePlot (canvas, context, filePath,
xColumnName,yColumnName, sizeColumnName,
minSize, maxSize, infoColumnNames, color,
canvasPadding,axesProperties)
```

```
{ ... }
```

Mapa de puntos

Uso



El método `drawMapDotPlot` dibuja un mapa tomando como base un archivo GeoJSON proporcionado, además grafica puntos específicos sobre él, tomando en cuenta el archivo csv indicado.

Entradas



- **canvas** (Canvas): Canvas.
- **context** (CanvasRenderingContext2D): Contexto.
- **mapDataFile** (String): Ruta al archivo GeoJSON
- **filePath** (String): Ruta al archivo de datos.
- **xColumnName** (String): Nombre de la columna de datos a usar para la longitud.
- **yColumnName** (String): Nombre de la columna de datos a usar para la latitud.
- **infoColumnNames** (Array[String]): Arreglo con el nombre de las columnas que se desplegarán como información al pasar el cursor sobre el gráfico.
- **color** (String): Color de los puntos.
- **filledCircles** (Boolean): True para círculos rellenos.
- **pointRadius** (number): Radio de los puntos.
- **canvasPadding** (number): Padding del canvas.

Salidas



No retorna valor.

Método { }

```
drawMapDotPlot(canvas, context,  
mapDataFile, filePath, xColumnName,  
yColumnName, infoColumnNames, color,  
filledCircles, pointRadius,  
canvasPadding)
```

```
{      ...      }
```

Mapa de burbujas

Uso



El método `drawMapBubblePlot` dibuja un mapa tomando como base un archivo GeoJSON proporcionado, además grafica burbujas sobre él tomando en cuenta el archivo csv indicado.

Entradas



- **canvas** (Canvas): Canvas.
- **context** (CanvasRenderingContext2D): Contexto.
- **mapDataFile** (String): Ruta al archivo GeoJSON
- **filePath** (String): Ruta al archivo de datos.
- **xColumnName** (String): Nombre de la columna de datos a usar para la longitud.
- **yColumnName** (String): Nombre de la columna de datos a usar para la latitud.
- **sizeColumnName** (String): Nombre de la columna que indica el tamaño de las burbujas.
- **minSizeBubble** (number): Tamaño mínimo de burbuja.
- **maxSizeBubble** (number): Tamaño máximo de burbuja.
- **infoColumnNames** (Array[String]): Arreglo con el nombre de las columnas que se desplegarán como información al pasar el cursor sobre el gráfico.
- **color** (String): Color de los puntos.
- **canvasPadding** (number): Padding del canvas.

Salidas



No retorna valor.

Método { }

```
drawMapDotPlot (canvas, context,  
mapDataFile, filePath, xColumnName,  
yColumnName, sizeColumnName,  
minSizeBubble, maxSizeBubble,  
infoColumnNames, color,  
canvasPadding)
```

```
{ ... }
```

Mapa de calor

Uso



El método `drawHeatMap` dibuja un mapa tomando como base un archivo GeoJSON proporcionado. Posteriormente hace uso de un archivo csv proporcionado por el usuario para colorear cada región del mapa en base a una variable determinada.

Entradas



- **canvas** (Canvas): Canvas.
- **canvasPadding** (number): Padding del canvas.
- **mapDataFile** (String): Ruta al archivo GeoJSON
- **csvFilePath** (String): Ruta al archivo de datos.
- **variableName** (String): Nombre de la columna de datos a usar para el degradado de color.
- **baseColor** (String): Color base para el degradado.
- **stateNameProperty** (String): Nombre de la propiedad que representa el identificador para cada región del mapa.
- **linkNameProperty** (String): Nombre de la columna en el archivo csv que identifica cada región del mapa y cuyos valores deben coincidir con los valores en `stateNameProperty`.
- **infoColumnNames** (Array[String]): Arreglo con el nombre de las columnas que se desplegarán como información al pasar el cursor sobre el gráfico.

Salidas



No retorna valor.

Método { }

```
drawHeatMap (canvas, canvasPadding,  
mapDataFile, csvFilePath,  
variableName, color,  
stateNameProperty, linkNameProperty,  
infoColumnNames)
```

```
{ ... }
```

Serie de tiempo

Uso



El método `drawTimeSeries` permite dibujar una serie de tiempo haciendo uso de curvas de bezier. Además, permite agregar eventos y representarlos mediante una banda de tiempo.

Entradas



- **canvas** (Canvas): Canvas.
- **context** (CanvasRenderingContext2D): Contexto.
- **filePath** (String): Ruta al archivo de datos.
- **xColumnName** (String): Nombre de la columna de fecha para usar en el eje de las abscisas.
- **yColumnName** (String): Nombre de la columna de valores referenciados a cada fecha.
- **infoColumnNames** (Array[String]): Arreglo con el nombre de las columnas que se desplegarán como información al pasar el cursor sobre el gráfico.
- **color** (String): color de la línea del gráfico.
- **filledCircles** (Boolean): True para círculos rellenos.
- **pointRadius** (number): Radio de los puntos.
- **lineWidth** (number): Grosor de la línea.
- **canvasPadding** (number): Padding del canvas.
- **axesProperties** (Object, opt): Objeto que contiene las propiedades definidas para los ejes (Ver caso práctico).
- **Events (Object,opt)** : Arreglo de eventos (Ver caso práctico)

Salidas



No retorna valor.

Método { }

```
drawTimeSeries(canvas, context, filePath,  
xColumnName,yColumnName, infoColumnNames,  
color, filledCircles, pointRadius,  
lineWidth, canvasPadding,  
axesProperties,events);  
  
{ ... }
```



Casos de uso

View Port

El siguiente ejemplo muestra cómo utilizar el método **initViewport** de la biblioteca vda.js.

viewPort.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>DataCanvas Ejemplo initView</title>
</head>
<body>
  <!--La siguiente línea define el canvas "miCanvas"-->
  <canvas id="miCanvas"></canvas>
  <script type="module" src="./main.js"></script>
</body>
</html>
```

El archivo viewPort.html define un elemento Canvas identificado por un ID con nombre "miCanvas".

Adicionalmente se tiene el archivo main.js el cual hace uso de la función initViewPort(), de la biblioteca vda.js, la cual recibe el id "miCanvas", y define los parámetros width y height con dimensiones 1000px cada uno. Posteriormente, para verificar el funcionamiento del código, dibujamos un rectángulo comenzando en el punto P(30,50) con ancho 2000 px y altura 500 px.

Main.js

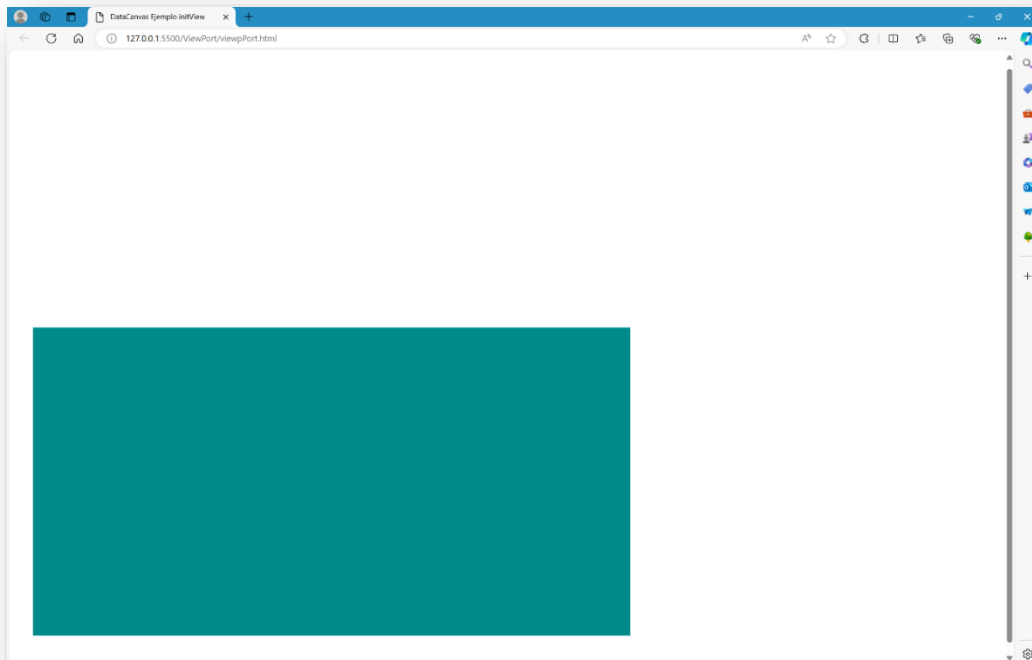
```
import {initViewport} from '../vda.js';

document.addEventListener('DOMContentLoaded', () => {

  // Llamamos a la función initViewPort de la biblioteca vda.js
  const width = 1000;
  const height = 1000;
  const context = initViewport('miCanvas', width, height);

  // Dibujamos rectángulo para verificar que el viewport funcion
  context.fillStyle = 'darkcyan';
  context.fillRect(30, 50, 2000, 500);
});
```

Navegador



Texto

El siguiente ejemplo muestra cómo utilizar el método **drawText** de la biblioteca vda.js.

text.html

```
<!DOCTYPE html>
<html lang="es">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>DataCanvas Ejemplo dibujar texto</title>
</head>

<body>
  <!--La siguiente línea define el canvas "miCanvas"-->
  <canvas id="miCanvas"></canvas>
  <script type="module" src="./main.js"></script>
</body>
```

El archivo text.html define un elemento Canvas identificado por un ID con nombre "miCanvas".

Adicionalmente se tiene el archivo main.js el cual hace uso de la función drawText(), de la biblioteca vda.js, la cual recibe el id "miCanvas", y define los parámetros width y height con dimensiones 1150 px y 1000px, respectivamente.

Posteriormente se definen los valores del texto como el texto mismo, color y tamaño; en este caso se está posicionando al texto en la parte superior de la pantalla y centrado con respecto al eje horizontal.

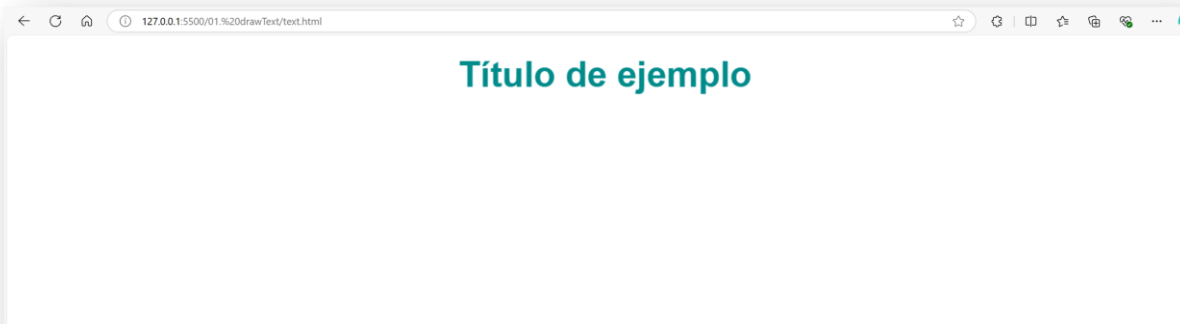
Main.js

```
// Llamamos a la función initViewPort de la biblioteca vda.js
const width = 1150;
const height = 10000;
const context = initViewPort('miCanvas', width, height);

const title = "Título de ejemplo";
const canvasPadding = 50;
const angleText = 0;
const size = 50;
const color = "darkcyan" ;
drawText(context,title, width/2 + canvasPadding ,height - canvasPadding,
size ,color,- angleText)

});
```

Navegador



Eje X

El siguiente ejemplo muestra cómo utilizar el método **drawXAxis** de la biblioteca vda.js.

Definimos un elemento canvas (miCanvas) donde se dibujarán los elementos gráficos.

viewPort.html

```
<!DOCTYPE html>
<html lang="es">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Canvas Ejemplo Eje X</title>
</head>

<body>
  <canvas id="miCanvas" style="border:1px solid #000000;"></canvas>
  <script type="module" src="./main.js"></script>
</body>

</html>
```

Posteriormente:

- Se importa el método `initViewport` y `drawXAxis` desde el archivo `main.js`.
- Cuando el documento está completamente cargado (`DOMContentLoaded`), inicializamos el canvas y su contexto mediante el método `initViewport()`.

- Llamamos a drawXAxis con los parámetros necesarios para dibujar el eje X en el canvas.

Main.js

```
import { initViewport, drawXAxis } from '../vda.js';

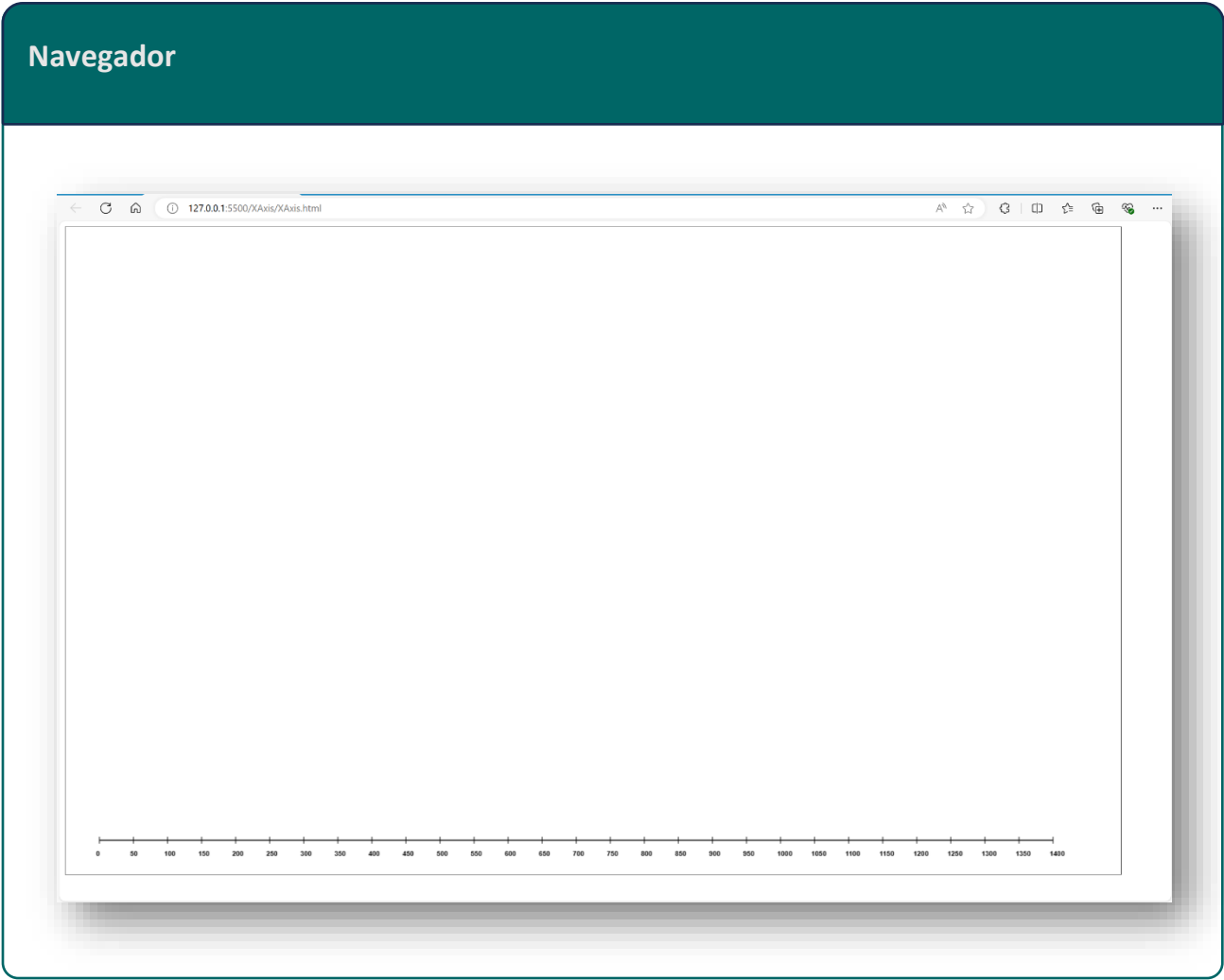
document.addEventListener('DOMContentLoaded', () => {
  const canvasId = 'miCanvas';
  const width = 1550;
  const height = 950;
  const context = initViewport(canvasId, width, height);

  // Configuración para el eje X
  const startX = 0;
  const endX = 1400;
  const step = 50;
  const xPosition = 50;
  const yPosition = 50;
  const color = 'black';
  const labelSpace = 20;

  drawXAxis(context, startX, endX, xPosition, yPosition, step, color,
labelSpace);
});
```

En este caso se está dibujando un eje X que inicia en 0 y finaliza en 1400, la posición del origen se encuentra en (50px, 50px), la separación en cada marca es de 50px y el espacio entre las leyendas (números de referencia en el eje) y el eje es de 20px.

En la siguiente imagen puede observarse el resultado, desplegado mediante un navegador web.



Eje Y

El siguiente ejemplo muestra cómo utilizar el método `drawYAxis` de la biblioteca `vda.js`.

Definimos un elemento `canvas` (`miCanvas`) donde se dibujarán los elementos gráficos.

viewPort.html

```
<!DOCTYPE html>
<html lang="es">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Canvas Ejemplo Eje Y</title>
</head>

<body>
  <canvas id="miCanvas" style="border:1px solid #000000;"></canvas>
  <script type="module" src="./main.js"></script>
</body>

</html>
```

Posteriormente:

- Se importa el método `initViewport` y `drawYAxis` desde el archivo `main.js`.
- Cuando el documento está completamente cargado (`DOMContentLoaded`), inicializamos el `canvas` y su contexto mediante el método `initViewport()`.

- Llamamos a drawYAxis con los parámetros necesarios para dibujar el eje X en el canvas.

Main.js

```
import { initViewport, drawYAxis } from '../vda.js';

document.addEventListener('DOMContentLoaded', () => {
  const canvasId = 'miCanvas';
  const width = 1550;
  const height = 950;
  const context = initViewport(canvasId, width, height);

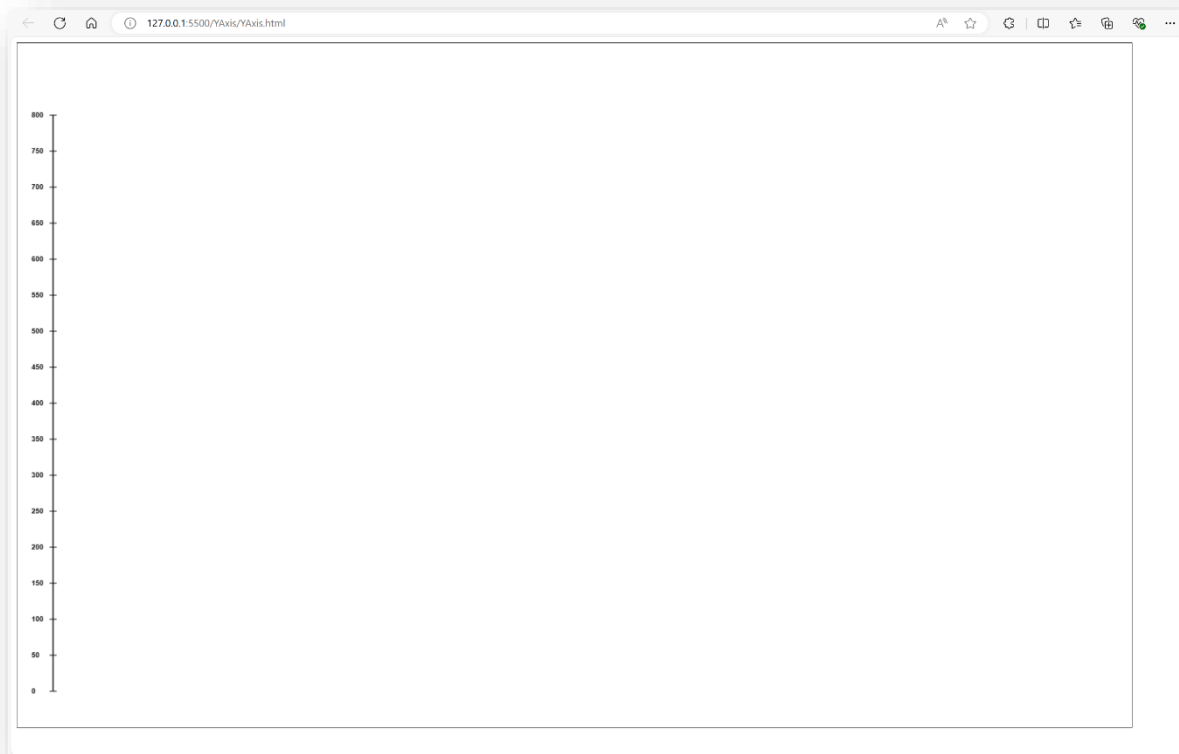
  // Configuración para el eje X
  const startY = 0;
  const endY = 800;
  const step = 50;
  const xPosition = 50;
  const yPosition = 50;
  const color = 'black';
  const labelSpace = 30;

  drawYAxis(context, startY, endY, xPosition, yPosition, step, color,
labelSpace);
});
```

En este caso se está dibujando un eje Y que inicia en 0 y finaliza en 800, la posición del origen se encuentra en (50px, 50px), la separación en cada marca es de 50px y el espacio entre las leyendas (números de referencia en el eje) y el eje es de 30px.

En la siguiente imagen puede observarse el resultado, desplegado mediante un navegador web.

Navegador



Eje X a partir de un arreglo de datos

El siguiente ejemplo muestra cómo utilizar el método `drawXAxisFromArray` de la biblioteca `vda.js`.

Definimos un elemento canvas (`miCanvas`) donde se dibujarán los elementos gráficos.

viewPort.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Ejemplo de Eje X a partir de un array</title>
  <script
src="https://cdnjs.cloudflare.com/ajax/libs/PapaParse/5.3.0/papaparse.
min.js"></script>
</head>
<body>
  <canvas id="miCanvas" width="1200" height="900" style="border:1px
solid #000000;"></canvas>
  <script type="module" src="./main.js"></script>
</body>
</html>
```

Posteriormente:

- Se importa el método `initViewport`, `drawXAxisWithCSV` y `loadCSV` desde el archivo `main.js`.
- Después, inicializamos el canvas y su contexto usando `initViewport()`.

- Cargamos los datos mediante la lectura del archivo csv haciendo uso de la función loadCSV().
- Finalmente, llamamos a drawXAxisFromArray con los parámetros necesarios para dibujar el eje X en el canvas.

Main.js

```
import { initViewPort, drawXAxisWithIntervals, loadCSV } from
'../vda.js';

// Función principal para inicializar el canvas y dibujar el eje X
async function init() {

    //Init viewPort
    const canvasId = 'miCanvas';
    const width = 1550;
    const height = 950;
    const context = initViewPort(canvasId, width, height);

    //Carga de datos desde CSV
    const data = await loadCSV("/data/xAxisData.csv");
    const xColumnName = "x"
    let xValues = data.map(row => row[xColumnName]);

    //Parametros para el eje X
    const xPos = 0;
    const yPos = 0;
    const color = "black";
    const labelSpace = 20;
    const canvasPadding = 50
    const xLabelTextAngle = 0;
    const interval = 0;

    // Llamar a la función para dibujar el eje X
    drawXAxisWithIntervals(context, xValues , xPos, yPos,
xLabelTextAngle, color, labelSpace, canvasPadding, interval);
}

// Llama a la función principal para inicializar todo
init();
```

En este caso, el arreglo de datos fue creado a partir del archivo `xAxisData.csv`, el cual contiene la una columna de datos con el encabezado "x". El archivo puede consultarse en la carpeta "data" del paquete "Vda".

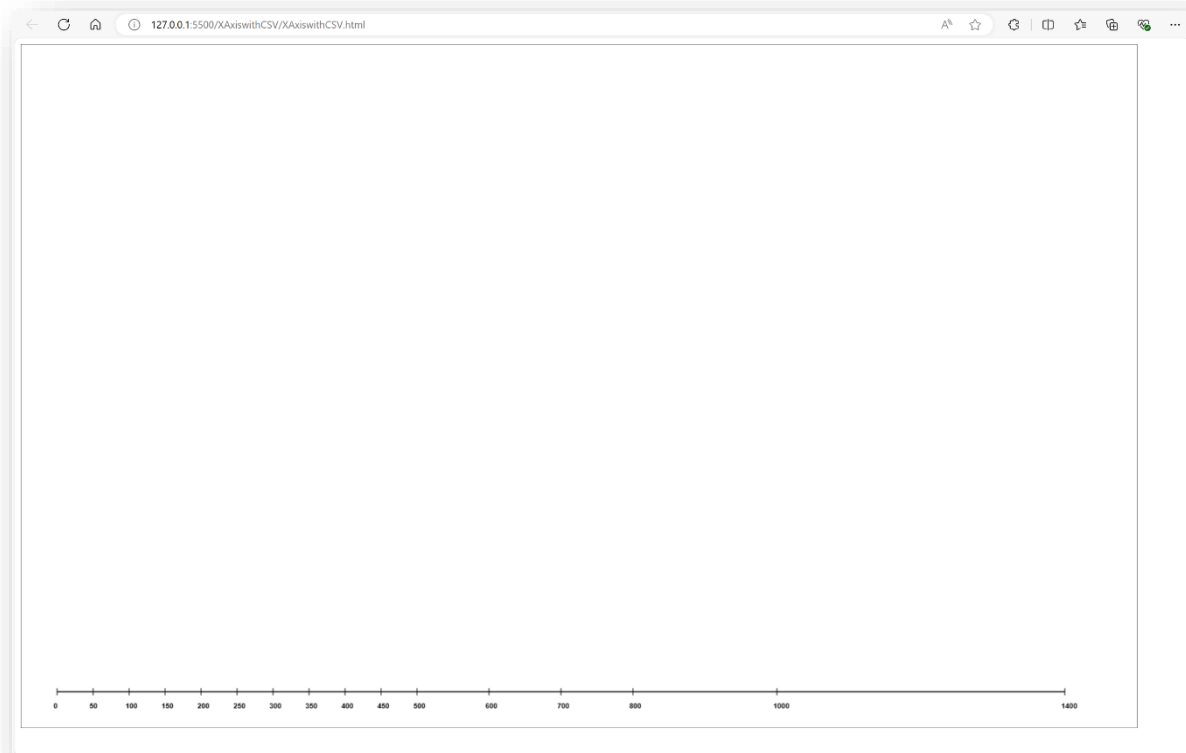
Nótese que el parámetro `Interval` está en 0, esto indica que los valores para los intervalos en el eje se tomaran directamente del arreglo de datos. Los valores aceptados para este parámetro son:

1. `> 0`: Cuando se coloca un valor mayor a cero, ese valor será el número de intervalos a definir.
2. `= 0`: Si el valor es igual a cero, se tomarán los valores del arreglo otorgado.
3. `-1`: Este valor indica que se calcule un intervalo de acuerdo a los datos otorgados.

Una vez desplegado, observamos un eje X que inicia en 0 y finaliza en 1400, y cuyos valores son tomados de la columna con nombre "x" del archivo csv cuya ruta es `'../data/xAxisData.csv'`.

La posición del origen se encuentra en (50 px, 50px), el color de la línea se indica negro y el espacio entre las leyendas (números de referencia en el eje) y el eje es de 20px.

Navegador



Eje Y a partir de un arreglo de datos

El siguiente ejemplo muestra cómo utilizar el método `drawYAxisFromArray` de la biblioteca `vda.js`.

Definimos un elemento canvas (`miCanvas`) donde se dibujarán los elementos gráficos.

viewPort.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Ejemplo de Eje Y a partir de un array</title>
  <script
src="https://cdnjs.cloudflare.com/ajax/libs/PapaParse/5.3.0/papaparse.
min.js"></script>
</head>
<body>
  <canvas id="miCanvas" style="border:1px solid #000000;"></canvas>
  <script type="module" src="./main.js"></script>
</body>
</html>
```

Posteriormente:

- Se importa el método `initViewport`, `drawYAxisWithCSV` y `loadCSV` desde el archivo `main.js`.
- Después, inicializamos el canvas y su contexto usando `initViewport()`.
- Cargamos los datos mediante la lectura del archivo csv haciendo uso de la función `loadCSV()`.

- Finalmente, llamamos a `drawYAxisFromArray` con los parámetros necesarios para dibujar el eje X en el canvas.

Main.js

```
import { initViewport, drawYAxisWithIntervals, loadCSV } from
'../vda.js';

// Función principal para inicializar el canvas y dibujar el eje X
async function init() {

    const canvasId = 'miCanvas';
    const width = 1550;
    const height = 950;
    const context = initViewport(canvasId, width, height);

    //Carga de datos desde CSV
    const data = await loadCSV("/data/yAxisData.csv");
    const yColumnName = "y";
    let yValues = data.map(row => row[yColumnName]);

    //Parametros para dibujar eje Y
    const yPosition = 0;
    const XPosition = 0;
    const color = "black";
    const labelSpace = -10;
    const canvasPadding = 50;
    const interval = 0;

    // Llamar a la función para dibujar el eje Y
    drawYAxisWithIntervals(context, yValues , XPosition, yPosition,
color, labelSpace, canvasPadding, interval);

}

// Llama a la función principal para inicializar todo
init();
```

En este caso, el arreglo de datos fue creado a partir del archivo `yAxisData.csv`, el cual contiene la una columna de datos con el encabezado "y". El archivo puede consultarse en la carpeta "data" del paquete "Vda".

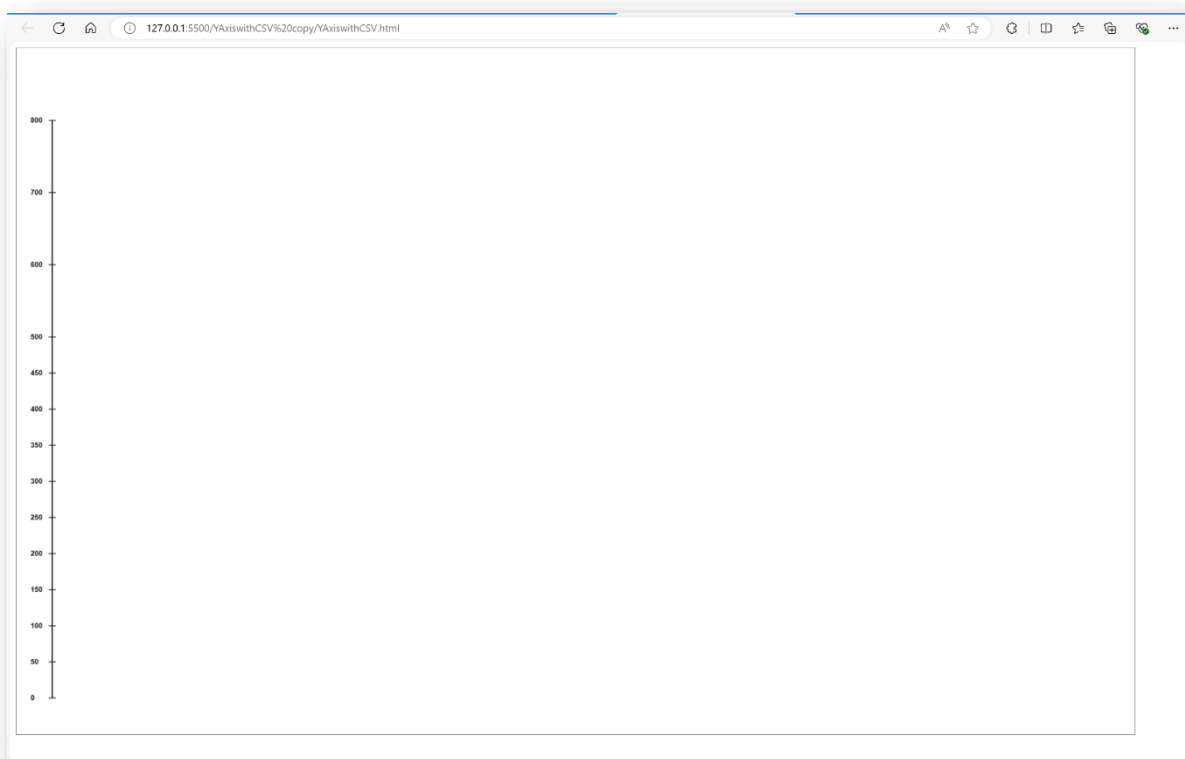
Nótese que el parámetro `Interval` está en 0, esto indica que los valores para los intervalos en el eje se tomaran directamente del arreglo de datos. Los valores aceptados para este parámetro son:

1. `> 0`: Cuando se coloca un valor mayor a cero, ese valor será el número de intervalos a definir.
2. `= 0`: Si el valor es igual a cero, se tomarán los valores del arreglo otorgado.
3. `-1`: Este valor indica que se calcule un intervalo de acuerdo a los datos otorgados.

Una vez desplegado, observamos un eje Y que inicia en 0 y finaliza en 800, y cuyos valores son tomados de la columna con nombre "y" del archivo csv cuya ruta es `'../data/yAxisData.csv'`.

La posición del origen se encuentra en (50px, 50px), el color de la línea se indica negro y el espacio entre las leyendas (números de referencia en el eje) y el eje es de 30px.

Navegador



Gráfica de puntos

El siguiente ejemplo muestra cómo utilizar el método `drawDotPlot` de la biblioteca `vda.js`.

Definimos un elemento `canvas` (`miCanvas`) donde se dibujarán los elementos gráficos.

viewPort.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Ejemplo Gráfica de puntos</title>
  <script
src="https://cdnjs.cloudflare.com/ajax/libs/PapaParse/5.3.0/papaparse.
min.js"></script>
  <link rel="stylesheet" href="./styles.css">
</head>
<body>
  <canvas id="miCanvas" width="1200" height="900" style="border:1px
solid #000000;"></canvas>
  <div id="infoOverlay" class="info-overlay"></div>
  <script type="module" src="./main.js"></script>
</body>
</html>
```

En este caso, también nos es posible agregar una etiqueta (de forma opcional)

```
<div id="infoOverlay"></div>
```

la cual nos permitirá agregar una capa de información desplegable cada vez que el mouse sea deslizado sobre el gráfico.

Posteriormente:

- Se importa los métodos `initViewport` , `drawDotPlot` y `loadCSV` desde el archivo `main.js`, y opcionalmente, `drawXAxisFromArray`, `drawYAxisFromArray`.
- Inicializamos el canvas y su contexto mediante el método `initViewport()`.
- Llamamos a `drawDotPlot` con los parámetros necesarios para el gráfico.

Main.js

```
import {initViewport,drawDotPlot,drawText,loadCSV} from '../vda.js';

document.addEventListener('DOMContentLoaded', async () => {

  //Init ViewPort
  const canvasId = 'miCanvas';
  const width = 1500;
  const height = 900;
  const context = initViewport(canvasId, width, height);

  //Carga de datos desde CSV
  const filePath = "/data/salarios.csv";
  const data = await loadCSV(filePath);

  //Parametros para la gráfica
  const canvas = document.getElementById("miCanvas");
  const xColumnName = "anios_estudio";
  const yColumnName = "sueldo_mensual";
  let infoColumnNames = [xColumnName, yColumnName, "edad"]
  const color = "darkcyan"
  const filledCircles = false;
  const pointRadius = 5;
  const canvasPadding = 50;

  //Parametros para los ejes
  const axesProperties = {
    xPos: 0,
    yPos: 0,
    xLabelSpace: 20,
    yLabelSpace: -10,
    xLabelTextAngle: 55,
    color: "black",
    xAxeType: 0.5,
    yAxeType: 500
  };

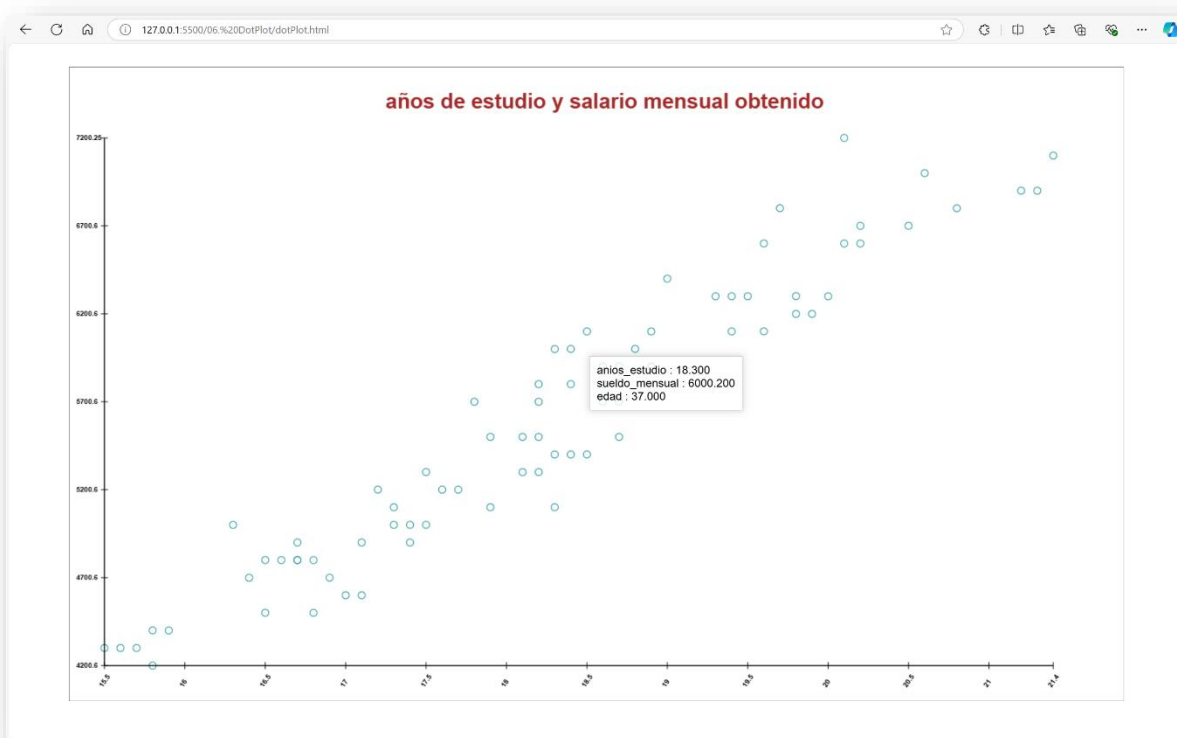
  drawDotPlot(canvas, context, filePath, xColumnName,yColumnName, infoColumnNames,
  color, filledCircles, pointRadius, canvasPadding,axesProperties);

  const xPos = width/2 - 6*canvasPadding ;
  const yPos = height -canvasPadding;
  const titleSize = 30;
  const titleColor = "brown"
  drawText(context,"años de estudio y salario mensual obtenido", xPos, yPos,
  titleSize ,titleColor,- 0)
});
```

Nótese que se hace uso del objeto `axesProperties`, el cual contiene los valores necesarios para dibujar los ejes X y Y del gráfico.

En la siguiente imagen podemos apreciar el gráfico de puntos dibujado en el navegador web.

Navegador



Gráfica de líneas

El siguiente ejemplo muestra cómo utilizar el método `drawLinePlot` de la biblioteca `vda.js`.

Definimos un elemento `canvas` (`miCanvas`) donde se dibujarán los elementos gráficos.

viewPort.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Ejemplo Gráfica de línea</title>
  <script
src="https://cdnjs.cloudflare.com/ajax/libs/PapaParse/5.3.0/papaparse.
min.js"></script>
  <link rel="stylesheet" href="./styles.css">
</head>
<body>
  <canvas id="miCanvas" width="1200" height="900" style="border:1px
solid #000000;"></canvas>
  <div id="infoOverlay" class="info-overlay"></div>
  <script type="module" src="./main.js"></script>
</body>
</html>
```

En este caso, también nos es posible agregar una etiqueta (de forma opcional)

```
<div id="infoOverlay"></div>
```

la cual nos permitirá agregar una capa de información desplegable cada vez que el mouse sea deslizado sobre el gráfico.

Posteriormente:

- Se importa los métodos `initViewport` , `drawLinePlot` y `loadCSV` desde el archivo `main.js`, y opcionalmente, `drawXAxisFromArray`, `drawYAxisFromArray`.
- Inicializamos el canvas y su contexto mediante el método `initViewport()`.
- Llamamos a `drawDotPlot` con los parámetros necesarios para el gráfico.

Main.js

```
import { initViewport, drawLinePlot, drawText, loadCSV } from '../vda.js';

document.addEventListener('DOMContentLoaded', async () => {

  //Init ViewPort
  const canvasId = 'miCanvas';
  const width = 1500;
  const height = 900;
  const context = initViewport(canvasId, width, height);

  //Carga de datos desde CSV
  const filePath = "/data/pesoEstatura.csv";
  const data = await loadCSV(filePath);

  //Parametros para la gráfica
  const canvas = document.getElementById("miCanvas");
  const xColumnName = "estatura";
  const yColumnName = "peso";
  let infoColumnNames = [xColumnName, yColumnName, "edad"];
  const color = "darkcyan";
  const filledCircles = true;
  const pointRadius = 10;
  const lineWidth = 1;
  const canvasPadding = 50;

  //Parametros para los ejes
  const axesProperties = {
    xPos: 0,
    yPos: 0,
    xLabelSpace: 20,
    yLabelSpace: -10,
    xLabelTextAngle: 20,
    color: "black",
    xAxeType: 0,
    yAxeType: 0
  };

  drawLinePlot(canvas, context, filePath, xColumnName, yColumnName, infoColumnNames,
    color, filledCircles, pointRadius, lineWidth, canvasPadding, axesProperties);

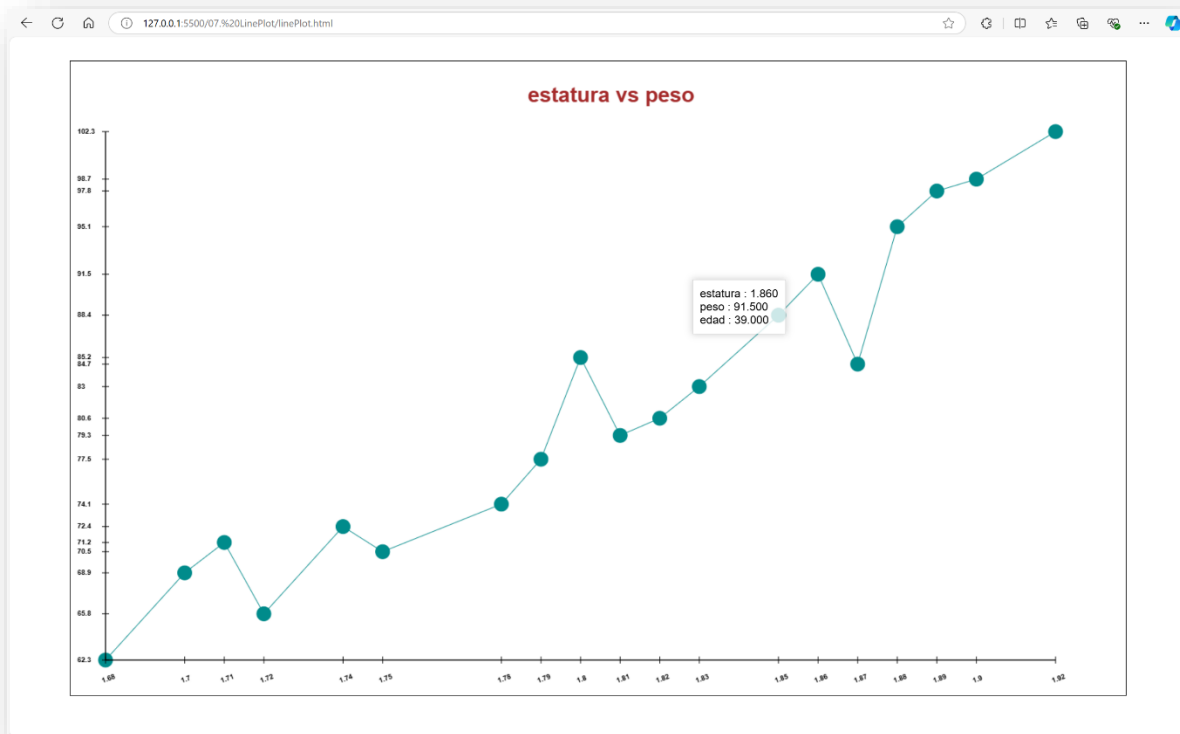
  const xPos = width/2 - 2*canvasPadding ;
  const yPos = height - canvasPadding;
  const titleSize = 30;
  const titleColor = "brown"
  drawText(context, "estatura vs peso", xPos, yPos, titleSize, titleColor, - 0)

});
```

Nótese que se hace uso del objeto `axesProperties`, el cual contiene los valores necesarios para dibujar los ejes X y Y del gráfico.

En la siguiente imagen podemos apreciar el gráfico de línea dibujado en el navegador web.

Navegador



Gráfica de barras

El siguiente ejemplo muestra cómo utilizar el método `drawBarPlot` de la biblioteca `vda.js`.

Definimos un elemento `canvas` (`miCanvas`) donde se dibujarán los elementos gráficos.

viewPort.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Ejemplo Gráfica de barras</title>
  <script
src="https://cdnjs.cloudflare.com/ajax/libs/PapaParse/5.3.0/papaparse.min.js"
></script>
  <link rel="stylesheet" href="./styles.css">
</head>
<body>
  <canvas id="miCanvas" width="1200" height="900" style="border:1px solid
#000000;"></canvas>
  <div id="infoOverlay" class="info-overlay"></div>
  <script type="module" src="./main.js"></script>
</body>
</html>
```

En este caso, también nos es posible agregar una etiqueta (de forma opcional)

```
<div id="infoOverlay"></div>
```

la cual nos permitirá agregar una capa de información desplegable cada vez que el mouse sea deslizado sobre el gráfico.

Posteriormente:

- Se importa los métodos `initViewport` , `drawBarPlot` y `loadCSV` desde el archivo `main.js`, y opcionalmente, `drawXAxisFromArray`, `drawYAxisFromArray`.
- Inicializamos el canvas y su contexto mediante el método `initViewport()`.
- Llamamos a `drawBarPlot` con los parámetros necesarios para el gráfico.

Main.js

```
import { initViewport, drawBarPlot, drawText, loadCSV } from
'../vda.js';

document.addEventListener('DOMContentLoaded', async () => {

  //Init ViewPort
  const canvasId = 'miCanvas';
  const width = 1500;
  const height = 900;
  const context = initViewport(canvasId, width, height);

  //Carga de datos desde CSV
  const filePath = "/data/perros.csv";
  const data = await loadCSV(filePath);

  //Parametros para la gráfica
  const canvas = document.getElementById("miCanvas");
  const xColumnName = "raza"
  const canvasPadding = 50;

  drawBarPlot(canvas, context, filePath, xColumnName,
canvasPadding);

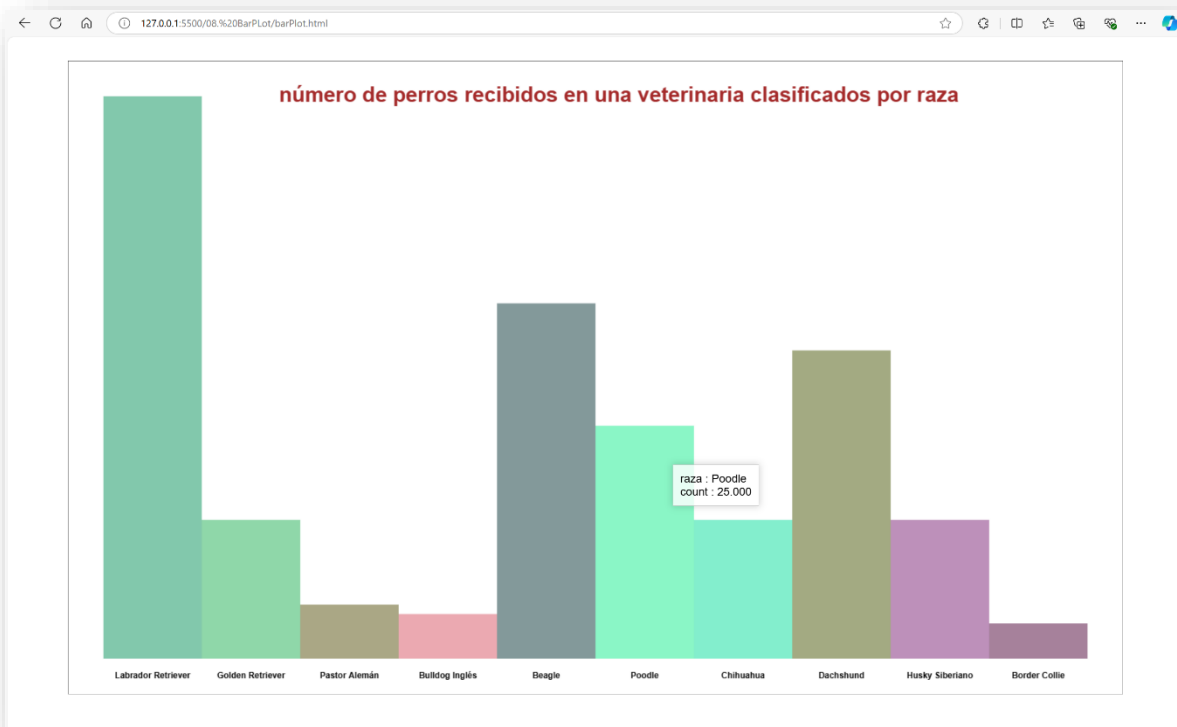
  const xPosTitle = width/2 - 9*canvasPadding ;
  const yPosTitle = height - canvasPadding;
  const titleSize = 30;
  const titleColor = "brown"

  drawText(context,"número de perros recibidos en una veterinaria
clasificados por raza", xPosTitle, yPosTitle, titleSize
,titleColor,- 0)

});
```


En la siguiente imagen podemos apreciar el gráfico de línea dibujado en el navegador web.

Navegador



Gráfica de burbujas

El siguiente ejemplo muestra cómo utilizar el método `drawBubblePlot` de la biblioteca `vda.js`.

Definimos un elemento `canvas` (`miCanvas`) donde se dibujarán los elementos gráficos.

viewPort.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Ejemplo Gráfica de burbujas</title>
  <script
src="https://cdnjs.cloudflare.com/ajax/libs/PapaParse/5.3.0/papaparse.min.js"
></script>
  <link rel="stylesheet" href="./styles.css">
</head>
<body>
  <canvas id="miCanvas" width="1200" height="900" style="border:1px solid
#000000;"></canvas>
  <div id="infoOverlay" class="info-overlay"></div>
  <script type="module" src="./main.js"></script>
</body>
</html>
```

En este caso, también nos es posible agregar una etiqueta (de forma opcional)

```
<div id="infoOverlay"></div>
```

la cual nos permitirá agregar una capa de información desplegable cada vez que el mouse sea deslizado sobre el gráfico.

Posteriormente:

- Se importa los métodos `initViewport` , `drawBubblePlot` y `loadCSV` desde el archivo `main.js`, y opcionalmente, `drawXAxisFromArray`, `drawYAxisFromArray`.
- Inicializamos el canvas y su contexto mediante el método `initViewport()`.
- Llamamos a `drawBubblePlot` con los parámetros necesarios para el gráfico.

Nótese que se hace uso del objeto `axesProperties`, el cual contiene los valores necesarios para dibujar los ejes X y Y del gráfico.

```
//Parametros para los ejes  
const axesProperties = {  
    xPos: 0,  
    yPos: 0,  
    xLabelSpace: 20,  
    yLabelSpace: -10,  
    xLabelTextAngle: 0,  
    color: "black",  
    xAxeType: -1,  
    yAxeType: -1  
};
```

Adicionalmente, hacemos uso de la función `drawText` para colocar un título y un subtítulo a la gráfica.

Título:

```
drawText(context, "Libros: precio vs número de páginas", xPosTitle, yPosTitle, titleSize ,titleColor, - 0)
```

Subtítulo:

```
drawText(context, "(El tamaño de burbuja equivale a la popularidad del libro)", xPosSubtitle, yPosSubtitle,  
subtitleSize ,titleColor, - 0)
```

Main.js

```
import {initViewport,drawBubblePlot,drawText,loadCSV} from '../vda.js';

document.addEventListener('DOMContentLoaded', async () => {

  //Init ViewPort
  const canvasId = 'miCanvas';
  const width = 1500;
  const height = 900;
  const context = initViewport(canvasId, width, height);

  //Carga de datos desde CSV
  const filePath = "/data/libros.csv";
  const data = await loadCSV(filePath);

  //Parametros para la gráfica
  const canvas = document.getElementById("miCanvas");
  const xColumnName = "precio";
  const yColumnName = "paginas";
  const zColumnName = "popularidad";
  let infoColumnNames = [xColumnName, yColumnName, zColumnName];
  const minSize = 5;
  const maxSize = 30;
  const transparence = 0.5;
  const color = 'rgba(0, 25, 215, 0.7)';
  const canvasPadding = 50;

  //Parametros para los ejes
  const axesProperties = {
    xPos: 0,
    yPos: 0,
    xLabelSpace: 20,
    yLabelSpace: -10,
    xLabelTextAngle: 0,
    color: "black",
    xAxeType: -1,
    yAxeType: -1
  };

  drawBubblePlot(canvas, context, filePath, xColumnName,yColumnName, zColumnName,
  minSize, maxSize, infoColumnNames, color, canvasPadding,axesProperties);

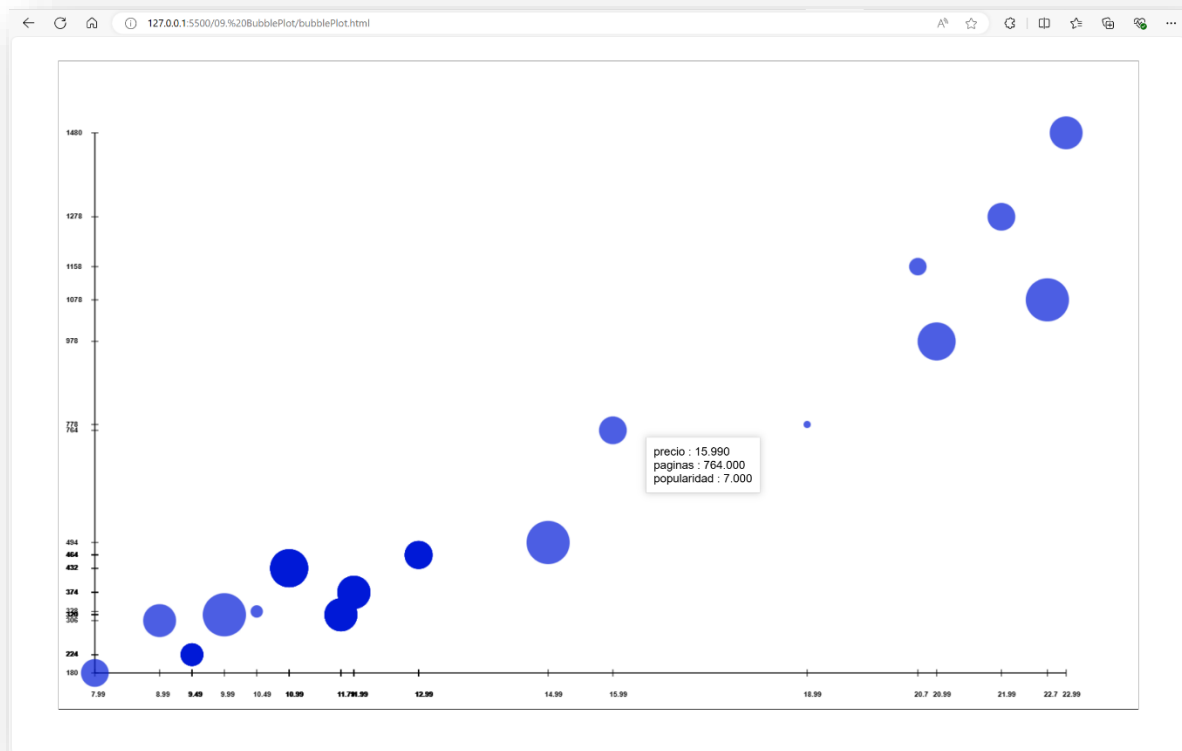
  const xPosTitle = width/2 - 5*canvasPadding ;
  const yPosTitle = height - canvasPadding;
  const titleSize = 30;
  const titleColor = "brown"
  const xPosSubtitle = width/2 - 4*canvasPadding ;
  const yPosSubtitle = height - 2*canvasPadding;
  const subtitleSize = 15;

  drawText(context,"Libros: precio vs número de páginas", xPosTitle, yPosTitle,
  titleSize ,titleColor,- 0)
  drawText(context,"(El tamaño de burbuja equivale a la popularidad del libro)",
  xPosSubtitle, yPosSubtitle, subtitleSize ,titleColor,- 0)

});
```

En la siguiente imagen podemos apreciar el gráfico de línea dibujado en el navegador web.

Navegador



Mapa de puntos

El siguiente ejemplo muestra cómo utilizar el método `drawMapDotPlot` de la biblioteca `vda.js`.

Definimos un elemento `canvas` (`miCanvas`) donde se dibujarán los elementos gráficos.

viewPort.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Ejemplo: Mapa de puntos</title>
  <script
src="https://cdnjs.cloudflare.com/ajax/libs/PapaParse/5.3.0/papaparse.min.js"
></script>
  <script src="https://d3js.org/d3.v6.min.js"></script>
  <link rel="stylesheet" href="./styles.css">
</head>
<body>
  <canvas id="miCanvas" width="1200" height="900" style="border:1px solid
#000000;"></canvas>
  <div id="infoOverlay" class="info-overlay"></div>
  <script type="module" src="./main.js"></script>
</body>
</html>
```

En este caso, también nos es posible agregar una etiqueta (de forma opcional)

```
<div id="infoOverlay"></div>
```

la cual nos permitirá agregar una capa de información desplegable cada vez que el mouse sea deslizado sobre el gráfico.

Posteriormente:

- Se importan los métodos `initViewport` y `drawMapDotPlot` desde el archivo `main.js`.
- Inicializamos el canvas y su contexto mediante el método `initViewport()`.
- Llamamos a `drawMapDotPlot` con los parámetros necesarios para el gráfico.

Main.js

```
import { initViewport, drawText, drawMapDotPlot } from '../vda.js';

document.addEventListener('DOMContentLoaded', async () => {

  //Init ViewPort
  const canvasId = 'miCanvas';
  const width = 1500;
  const height = 900;
  const context = initViewport(canvasId, width, height);

  const canvas = document.getElementById(canvasId);
  const canvasPadding = 50;
  const mapDataFile = "/data/states.geojson"

  //Carga de datos desde CSV
  const filePath = "/data/puntos_mapa.csv";

  //Parametros para la gráfica
  const longitudeColumnName = "longitud"
  const latitudeColumnName = "latitud";
  let infoColumnNames=[longitudeColumnName,latitudeColumnName,"name"]
  const color = "darkcyan"
  const filledCircles = true;
  const pointRadius = 7;

  drawMapDotPlot(canvas, context, mapDataFile, filePath,
  longitudeColumnName, latitudeColumnName, infoColumnNames, color,
  filledCircles, pointRadius, canvasPadding)

  const xPos = width/2 - 8*canvasPadding ;
  const yPos = height -canvasPadding;
  const titleSize = 30;
  const titleColor = "brown"
  drawText(context,"Algunas ciudades de México y sus localización
  geográfica", xPos, yPos, titleSize ,titleColor,- 0)

});
```

En la siguiente imagen podemos apreciar el gráfico de línea dibujado en el navegador web.

Navegador



Mapa de burbujas

El siguiente ejemplo muestra cómo utilizar el método `drawMapBubblePlot` de la biblioteca `vda.js`.

Definimos un elemento canvas (`miCanvas`) donde se dibujarán los elementos gráficos.

viewPort.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Ejemplo: Mapa de burbujas</title>
  <script
src="https://cdnjs.cloudflare.com/ajax/libs/PapaParse/5.3.0/papaparse.min.js"
></script>
  <script src="https://d3js.org/d3.v6.min.js"></script>
  <link rel="stylesheet" href="./styles.css">
</head>
<body>
  <canvas id="miCanvas" width="1200" height="900" style="border:1px solid
#000000;"></canvas>
  <div id="infoOverlay" class="info-overlay"></div>
  <script type="module" src="./main.js"></script>
</body>
</html>
```

En este caso, también nos es posible agregar una etiqueta (de forma opcional)

```
<div id="infoOverlay"></div>
```

la cual nos permitirá agregar una capa de información desplegable cada vez que el mouse sea deslizado sobre el gráfico.

Posteriormente:

- Se importan los métodos `initViewport` y `drawMapBubblePlot` desde el archivo `main.js`.
- Inicializamos el canvas y su contexto mediante el método `initViewport()`.
- Llamamos a `drawMapBubblePlot` con los parámetros necesarios para el gráfico.

Main.js

```
import { initViewport, drawText, drawMapBubblePlot } from '../vda.js';

document.addEventListener('DOMContentLoaded', async () => {

  //Init ViewPort
  const canvasId = 'miCanvas';
  const width = 1500;
  const height = 900;
  const context = initViewport(canvasId, width, height);

  const canvas = document.getElementById(canvasId);
  const canvasPadding = 50;
  const mapDataFile = "/data/states.geojson";

  //Carga de datos desde CSV
  const csvFilePath = "/data/bubbles_mapa.csv";

  //Parametros para la gráfica
  const longitudeColumnName = "longitud";
  const latitudeColumnName = "latitud";
  const sizeColumnName = "poblacion";
  const minSizeBubble = 2;
  const maxSizeBubble = 50;
  let infoColumnNames = [longitudeColumnName, latitudeColumnName, "name" ,
    "poblacion"];
  const color = "darkcyan";

  drawMapBubblePlot(canvas, context, mapDataFile, csvFilePath, longitudeColumnName,
    latitudeColumnName, sizeColumnName, minSizeBubble, maxSizeBubble, infoColumnNames,
    color, canvasPadding)

  const xPos = width/2 - 8*canvasPadding ;
  const yPos = height - canvasPadding;
  const titleSize = 30;
  const titleColor = "brown"
  const xPosSubtitle = width/2 - 3.5*canvasPadding ;
  const yPosSubtitle = height - 2*canvasPadding;
  const subtitleSize = 15;

  drawText(context, "Algunas ciudades de México y su localización geográfica", xPos,
    yPos, titleSize ,titleColor,- 0)
  drawText(context, "(El tamaño de burbuja equivale al número de habitantes)",
    xPosSubtitle, yPosSubtitle, subtitleSize ,titleColor,- 0)

});
```

En la siguiente imagen podemos apreciar el gráfico de línea dibujado en el navegador web.

Navegador



Mapa de color (República Mexicana)

El siguiente ejemplo muestra cómo utilizar el método `drawHeatMap` de la biblioteca `vda.js`.

Definimos un elemento `canvas` (`miCanvas`) donde se dibujarán los elementos gráficos.

viewPort.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Ejemplo: Mapa de color México</title>
  <script
src="https://cdnjs.cloudflare.com/ajax/libs/PapaParse/5.3.0/papaparse.min.js"
></script>
  <script src="https://d3js.org/d3.v6.min.js"></script>
  <link rel="stylesheet" href="./styles.css">
</head>
<body>
  <canvas id="miCanvas" width="1200" height="900" style="border:1px solid
#000000;"></canvas>
  <div id="infoOverlay" class="info-overlay"></div>
  <script type="module" src="./main.js"></script>
</body>
</html>
```

En este caso, también nos es posible agregar una etiqueta (de forma opcional)

```
<div id="infoOverlay"></div>
```

la cual nos permitirá agregar una capa de información desplegable cada vez que el mouse sea deslizado sobre el gráfico.

Posteriormente:

- Se importan los métodos `initViewport` y `drawHeatMap` desde `main.js`.
- Inicializamos el canvas y su contexto mediante el método `initViewport()`.
- Proporcionamos los archivos GeoJson y CSV para la geometría del mapa y los datos a representar respectivamente.
- Definimos los valores `linkNameProperty`, `stateNameProperty`, que son los identificadores de las columnas que comparten ambos sets de datos y que sirven como llaves únicas para enlazar ambos.
- Llamamos a `drawHeatMap` con los parámetros necesarios para el gráfico.

Main.js

```
import { initViewport, drawText, drawHeatMap } from '../vda.js';

document.addEventListener('DOMContentLoaded', async () => {

  //Init ViewPort
  const canvasId = 'miCanvas';
  const width = 1500;
  const height = 900;
  const context = initViewport(canvasId, width, height);

  //Obtención de polígonos para dibujar
  const canvas = document.getElementById(canvasId);
  const canvasPadding = 50;
  const mapDataFile = "/data/states.geojson";

  // Parámetros para el heatMap
  const csvFilePath = '/data/colorMap_Mexico.csv';
  const variableName = 'urbanizacion'; // Nombre de la columna en el CSV
  //const variableName = 'poblacion'; // Nombre de la columna en el CSV
  //const variableName = 'temperatura'; // Nombre de la columna en el CSV
  const baseColor = "purple"
  const stateNameProperty="state_name";//nombre de la propiedad en archivo geojson
  const linkNameProperty = "nombre"; //nombre de la columna con la que se comparará
  el stateNameProperty
  let infoColumnNames = ["nombre", "temperatura", "poblacion", "urbanizacion"];

  drawHeatMap(canvas, canvasPadding, mapDataFile, csvFilePath, variableName,
  baseColor, stateNameProperty, linkNameProperty, infoColumnNames);

  const xPos = width/2 - 4*canvasPadding ;
  const yPos = height -canvasPadding;
  const titleSize = 30;const titleColor = "brown";
  const xPosSubtitle = width/2 - 4.2*canvasPadding ;
  const yPosSubtitle = height - 1.8*canvasPadding;
  const subtitleSize = 15;

  drawText(context,"Estados de la República Mexicana",xPos,yPos,titleSize
  ,titleColor,- 0);
  drawText(context,"(El degradado de color indica el grado de urbanización en cada
  estado)", xPosSubtitle, yPosSubtitle, subtitleSize ,titleColor,- 0);

});
```

En la siguiente imagen podemos apreciar el gráfico de línea dibujado en el navegador web.

Navegador



Mapa de color (USA)

Este ejemplo sirve para enfatizar que es posible indicar un archivo GeoJSON propio y no únicamente los dos establecidos en estos casos de uso; esto permite dibujar cualquier geometría siempre y cuando se respete la estructura de este tipo de archivos.

Definimos un elemento canvas (miCanvas) donde se dibujará el mapa de calor.

viewPort.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Ejemplo: Mapa de color USA</title>
  <script
src="https://cdnjs.cloudflare.com/ajax/libs/PapaParse/5.3.0/papaparse.min.js"
></script>
  <script src="https://d3js.org/d3.v6.min.js"></script>
  <link rel="stylesheet" href="./styles.css">
</head>
<body>
  <canvas id="miCanvas" width="1200" height="900" style="border:1px solid
#000000;"></canvas>
  <div id="infoOverlay" class="info-overlay"></div>
  <script type="module" src="./main.js"></script>
</body>
</html>
```

En este caso, también nos es posible agregar una etiqueta (de forma opcional)

```
<div id="infoOverlay"></div>
```

la cual nos permitirá agregar una capa de información desplegable cada vez que el mouse sea deslizado sobre el gráfico.

Posteriormente:

- Se importan los métodos `initViewport` y `drawHeatMap` desde `main.js`.
- Inicializamos el canvas y su contexto mediante el método `initViewport()`.
- Proporcionamos los archivos GeoJson y CSV para la geometría del mapa y los datos a representar respectivamente.
- Definimos los valores `linkNameProperty`, `stateNameProperty`, que son los identificadores de las columnas que comparten ambos sets de datos y que sirven como llaves únicas para enlazar ambos.
- Llamamos a `drawHeatMap` con los parámetros necesarios para el gráfico.

Main.js

```
import { initViewport, drawText, drawHeatMap } from '../vda.js';

document.addEventListener('DOMContentLoaded', async () => {

  //Init ViewPort
  const canvasId = 'miCanvas';
  const width = 1500; const height = 900;
  const context = initViewport(canvasId, width, height);

  //Obtención de polígonos para dibujar
  const canvasPadding = 50;
  const mapDataFile = "/data/usaStates_simple.geojson"

  // Parámetros para el heatMap
  const canvas = document.getElementById(canvasId);
  const csvFilePath = '/data/colorMap_USA.csv';
  //const variableName = 'urbanizacion'; // Nombre de la columna en el CSV
  //const variableName = 'poblacion'; // Nombre de la columna en el CSV
  const variableName = 'temperatura'; // Nombre de la columna en el CSV
  const stateNameProperty = "name" //nombre de la propiedad en archivo geojson
  const linkNameProperty = "nombre" //nombre de la columna con la que se
  comparará el stateNameProperty
  let infoColumnNames=["nombre","temperatura","poblacion","urbanizacion"];

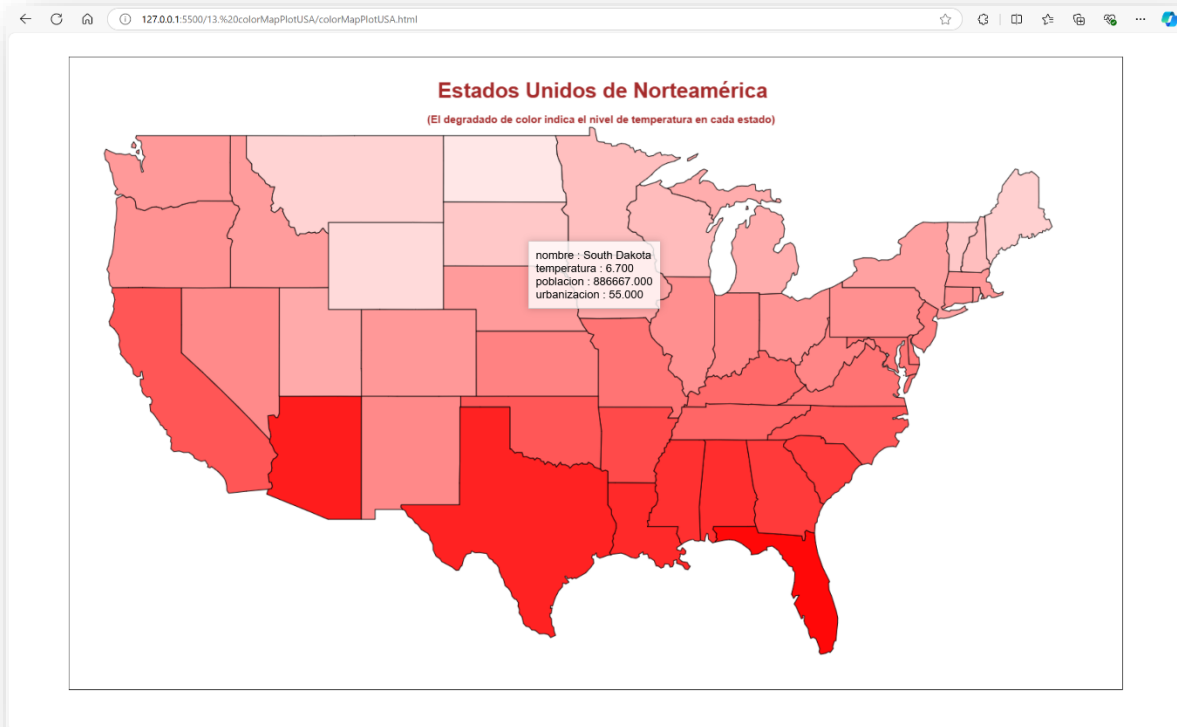
  drawHeatMap(canvas, canvasPadding, mapDataFile, csvFilePath, variableName,
  "red", stateNameProperty, linkNameProperty, infoColumnNames);

  const xPos = width/2 - 4.5*canvasPadding ;
  const yPos = height - canvasPadding;
  const titleSize = 30; const titleColor = "brown";
  const xPosSubtitle = width/2 - 4.8*canvasPadding ;
  const yPosSubtitle = height - 1.8*canvasPadding; const subtitleSize = 15;

  drawText(context,"Estados Unidos de Norteamérica", xPos, yPos, titleSize
  ,titleColor,- 0);
  drawText(context,"(El degradado de color indica el nivel de temperatura en
  cada estado)", xPosSubtitle, yPosSubtitle, subtitleSize ,titleColor,- 0);
});
```


En la siguiente imagen podemos apreciar el gráfico de línea dibujado en el navegador web.

Navegador



Serie de tiempo

En este ejemplo se hace uso de una serie de eventos cronológicos. Se dibuja una colección de puntos con base en el conjunto de fechas dadas en el formato yyyy-mm-dd y un valor asociado a cada una de ellas. El valor en las abscisas será definido por la fecha, mientras que el valor en las ordenadas estará definido por el valor asociado a cada fecha; el archivo con estos datos puede obtenerse de la carpeta `data time_series_100.csv` del repositorio de GitHub.

Mediante el método `drawTimeSeries` podremos dibujar las líneas que conectan los puntos en la serie de tiempo, haciendo uso de curvas de Bezier. Las curvas de Bezier son un tipo de curva paramétrica ideal para modelar formas suaves, se definen mediante un conjunto de puntos de control y son extremadamente flexibles.

Existen curvas de bezier lineales, cuadráticas y cúbicas; la librería VDA hace uso de curvas de bezier cuadráticas para dibujar las líneas entre los puntos, estas curvas están definidas por cuatro puntos: un punto inicial

Como se puede ver en el código del archivo `mai.js`, utilizaremos el método `bezierCurveTo`, el cual es parte de la API de dibujo en canvas de HTML5, el cual se define de la siguiente manera:

`bezierCurveTo(cp1x,cp1y,cp2x,cp2y,x,y)`

donde:

1. `cp1x, cp1y`: Coordenadas del primer punto de control
2. `cp2x, cp2y`: Coordenadas del segundo punto de control
3. `x, y`: Coordenadas del punto final

Notar que el punto inicial es el último punto que se dibujó.

timeSeriesPlot.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Ejemplo Gráfica línea de tiempo</title>
  <script
src="https://cdnjs.cloudflare.com/ajax/libs/PapaParse/5.3.0/papaparse.min.js"
></script>
  <link rel="stylesheet" href="./styles.css">
</head>
<body>
  <canvas id="miCanvas" width="1200" height="900" style="border:1px solid
#000000;"></canvas>
  <div id="infoOverlay" class="info-overlay"></div>
  <div id="eventToolTip" style="position: absolute; display: none;
background: rgba(0, 0, 0, 0.7); color: white; padding: 5px; border-radius:
5px;"></div>
  <script type="module" src="./main.js"></script>
</body>
</html>
```

Adicionalmente, el método permite agregar una serie de bandas de tiempo para enfatizar eventos dentro de la serie de tiempo. Estos eventos son enviados como un objeto de array al método, tal como puede observarse en el código del archivo main.js

Para visualizar los eventos, será necesario incluir la siguiente etiqueta en el archivo timeSeriesPlot.html

```
<div id="eventTootlTip"></div>
```

la cual nos permitirá agregar una capa de información desplegable cada vez que el mouse sea deslizado sobre el título de cada evento.

Finalmente, en el archivo main.js:

- Se importan los métodos initViewport y drawTimeSeries desde main.js.
- Inicializamos el canvas y su contexto mediante el método initViewport().
- Proporcionamos el archivo CSV que contiene la serie de tiempo.
- Definimos parámetros para la gráfica, ejes en el plano y array de eventos.
- Llamamos a drawTimeSeries para dibujar el gráfico.

Main.js

```
import { initViewport, drawTimeSeries, drawText, loadCSV } from '../vda.js';

document.addEventListener('DOMContentLoaded', async () => {

  //Init ViewPort
  const canvasId = 'miCanvas';
  const width = 1500; const height = 900;
  const context = initViewport(canvasId, width, height);

  //Carga de datos desde CSV
  const filePath = "/data/time_series_100.csv";
  const data = await loadCSV(filePath);

  //Parametros para la gráfica
  const canvas = document.getElementById("miCanvas");
  const xColumnName = "date";
  const yColumnName = "value";
  let infoColumnNames = [xColumnName, yColumnName];
  const color = "darkcyan";
  const filledCircles = true; const pointRadius = 10;
  const lineWidth = 1;
  const canvasPadding = 50;

  //Parametros para los ejes
  const axesProperties = {
    xPos: 0,
    yPos: 0,
    xLabelSpace: 2,
    yLabelSpace: -10,
    xLabelTextAngle: 60,
    color: "black",
    xAxeType: -1,
    yAxeType: -1
  };

  //Eventos definidos para la serie de tiempo
  const events = [
    { startDate: '2023-01-13', endDate: '2023-02-10', title: 'Título evento 2', desc:
'Descripción 2', color: 'rgba(20, 0, 20, 0.3)' },
    { startDate: '2023-03-01', endDate: '2023-03-15', title: 'Título evento 1', desc:
'Descripción 1', color: 'rgba(105, 0, 123, 0.2)' },
    { startDate: '2023-04-03', endDate: '2023-04-03', title: 'Título evento 2', desc:
'Descripción 2', color: 'rgba(220, 0, 20, 0.3)' }
  ];

  // Llamada a la función para dibujar la serie de tiempo
  drawTimeSeries(canvas, context, filePath, xColumnName, yColumnName, infoColumnNames, color,
filledCircles, pointRadius, lineWidth, canvasPadding, axesProperties, events);

  const xPos = width/2 - 2*canvasPadding ;
  const yPos = height - canvasPadding;
  const titleSize = 30;
  const titleColor = "brown"
  drawText(context, "Serie de tiempo", xPos, yPos, titleSize, titleColor, - 0)
});
```

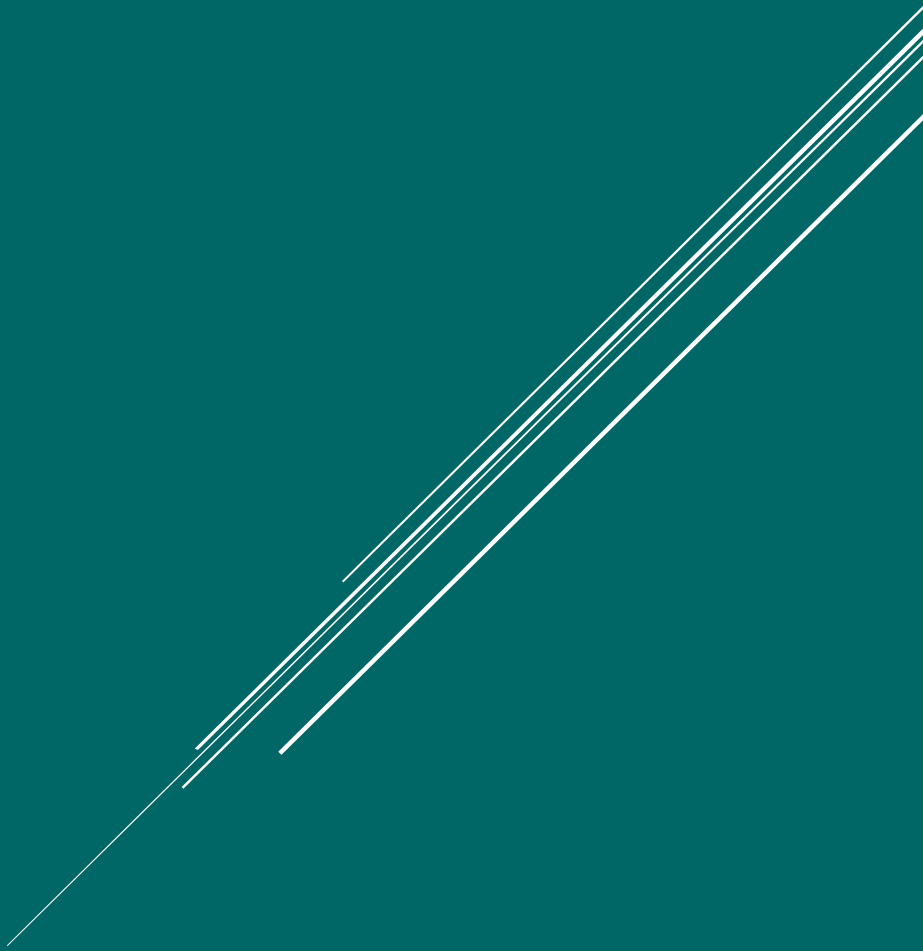
En la siguiente imagen podemos apreciar el gráfico de línea dibujado en el navegador web.

Navegador



Referencias

Repositorio GitHub. El código para implementar los casos de uso, así como los archivos de datos csv y geoJson utilizados en esta guía, pueden obtenerse directamente del siguiente [repositorio](#).



VDA

Librería para Visualización de Datos

García Aguilar Luis Alberto